

ApproxDPPoint::beta

Aishwarya Ramasethu (@aramasethu), Yu-Ju Ku (@yuju1998), Jordan Awan, Michael Shoemate (@Shoeboxam)

August 18, 2026

This proof resides in “contrib” because it has not completed the vetting process.

1 Hoare Triple

Precondition

Compiler-verified

- Receiver `self` of type `ApproxDPPoint`
- Argument `alpha` of type `RBig`

Caller-verified

`alpha` is in $[0, 1]$, and `self` was constructed by `ApproxDPPoint::build` from finite nonnegative ϵ and $\delta \in [0, 1]$.

Pseudocode

```
1 def beta(  
2     self: ApproxDPPoint,  
3     alpha: RBig,  
4 ) -> RBig:  
5     t1 = self.one_minus_delta - self.exp_eps_up * alpha  
6     base = max(self.one_minus_delta - alpha, RBig(0))  
7     t2 = self.exp_neg_eps_down * base  
8     return max(max(t1, t2), RBig(0))
```

Postcondition

Theorem 1.1. For all $\alpha \in [0, 1]$, `ApproxDPPoint::beta` returns a value no greater than the exact approximate-DP tradeoff curve $f_{\epsilon, \delta}(\alpha)$.

Definition 1.2. For an `ApproxDPPoint` with parameters ϵ and δ , define

$$f_{\epsilon, \delta}(\alpha) = \max(0, 1 - \delta - \exp(\epsilon)\alpha, \exp(-\epsilon) \max(1 - \delta - \alpha, 0)). \quad (1)$$

Proof. The receiver was constructed by `ApproxDPPoint::build`, which stores exact rational conversions of directed interval endpoints satisfying

$$\text{self.exp_eps_up} \geq \exp(\epsilon), \quad \text{self.exp_neg_eps_down} \leq \exp(-\epsilon).$$

The pseudocode uses these cached constants with exact rational arithmetic.

For $\alpha \in [0, 1]$, the first affine branch is no greater than $1 - \delta - \exp(\epsilon)\alpha$ because `self.exp_eps_up` is an upper bound and α is nonnegative. The second branch is similarly no greater than its exact counterpart because `self.exp_neg_eps_down` is a lower bound. Taking the maximum with zero preserves the pointwise lower bound, so the returned value is a conservative tradeoff guarantee. $\square$